

From Natural Language Policies to Executable Obligations: A Verification Harness for Dependable In-Car LLM Agents

Radouane Boucheikir, Damir Safin and Tomas Bueno Momcilovic

fortiss GmbH, Munich, Germany

{boucheikir, safin, momcilovic}@fortiss.org

Abstract

Large Language Models (LLMs) agents deployed in vehicles must satisfy a written operating policy on every turn: a single hallucinated identifier, omitted mandatory side-effect, or premature completion claim fails the task. We present `AgentGuardUtil`, our entry to CAR-bench Track 1, which treats the AI planner (LLM) as a fallible proposer inside a grounded verify-and-revise loop. Its core novelty is a *runtime policy compiler*: the natural-language policy shipped with each conversation is compiled, once per policy, into typed machine-checkable rules, a subset of which receive an *executable* form. A deterministic obligation engine interprets these rules against live tool results and the simulated post-write state of the draft itself, emitting the exact remedial calls with computed arguments rather than natural-language reminders. Around this engine, 25 deterministic gates (*identifier provenance*, *schema and enum validity*, *gather-before-act*, *confirmation and future-time protocols*) and an LLM critic produce tiered findings that drive a bounded revision loop tuned for the Pass^k metric.

1 Introduction

LLM-based agents are moving rapidly from research prototypes into products that act on the physical world. In-vehicle voice assistants are an exemplary setting: the agent controls windows, climate, lighting, navigation, charging, and communication on behalf of a driver whose attention is limited, under a manufacturer policy that encodes safety and interaction rules. Therefore, the most impactful scenario involving an unreliable agent is an autonomous action (e.g., a wrong interpretation of a speed limits) that leads to a safety incident (e.g., crash into another vehicle) - far more than a lower benchmark point.

The conditions under which current AI agents are evaluated differ substantially from those encountered in deployment. In operational settings, user requests are frequently incomplete or ambiguous, requiring agents to either infer missing information or request clarification. Necessary tools, parameters, or external resources may be unavailable, making it essential for agents to acknowledge these limitations rather than fabricate outcomes. In addition, deployment environments impose domain-specific policies that regulate how

tasks may be performed, including preconditions, mandatory side effects, confirmation requirements, and disclosure obligations. CAR-bench [4] captures these deployment challenges in the in-car domain. The benchmark evaluates ambiguity resolution (*disambiguation*), admitting unavailable capabilities (*hallucination*), and policy-compliant execution (*base*), using conjunctive success criteria over environment state, tool execution, policy compliance, and task termination.

In this paper we present `AgentGuardUtil`, our submission to CAR-bench Track 1¹. The design rests on two complementary principles (see Fig. 1). First, a *student-teacher* decomposition separates proposal from approval: the LLM (student) only *proposes* actions, while a verification harness (teacher) reviews each draft, returns concrete findings, and re-verifies the revision before any action reaches the agent (simulated driver). Second, the interaction between the two follows an *assume-guarantee* discipline in the spirit of compositional verification [2]: the verification harness assumes only that the student is a capable but fallible proposer, and carries the *guarantee* (i.e. policy compliance, grounding in observed data, and policy adherence) through deterministic checks compiled from the conversation’s own artifacts (policy text, tool schemas, observations), independently of the student’s behavior. At the first turn, the natural-language policy is compiled into typed machine-checkable rules, part of which receive an *executable* form. On every draft action, a deterministic obligation engine evaluates these rules against live tool results and the simulated post-write state of the draft itself, computing the exact remedial calls the policy requires; 25 further deterministic gates enforce identifier provenance, schema validity, and interaction protocols, and an LLM critic adjudicates the uncertain findings. Surviving findings drive a bounded revise-and-recheck loop before anything reaches the simulated driver. The main contributions of this work are summarized as follows: (i) **Runtime policy compiler** producing typed rules with an executable sub-language, cached per policy hash (§2.2); (ii) **Deterministic obligation engine** that evaluates executable rules on a projection of the environment state, including *simulation of the draft’s own writes*, and emits exact remedial tool calls (§2.3); (iii) **Tiered verification loop** combining deterministic gates

¹<https://car-bench.github.io/car-bench/index.html>

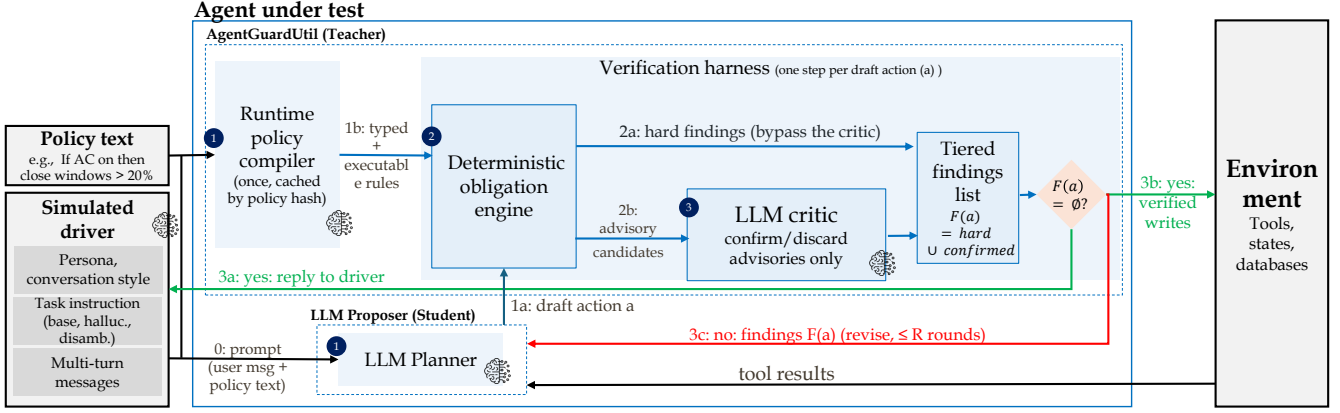


Figure 1: Overview of AgentGuardUtil architecture and one verified turn. A student LLM proposes; a deterministic teacher disposes. The policy is compiled once into executable rules (1). Each draft action (2) passes one verification step: deterministic gates, including the obligation engine, emit hard findings, and an LLM critic may only confirm or discard advisory candidates (3). If the tiered list $F(a)$ is non-empty, the draft returns to the student for revision, $\leq R$ rounds; if empty, verified writes reach the environment and the reply reaches the simulated drive. Under the assume–guarantee reading, the student is assumed only capable-but-fallible; every arrow leaving the teacher carries the guarantee.

with an LLM critic (§2.4). The design composes chain-of-verification prompting [3] and self-consistency [1], but moves the decisive checks out of the model and into code that the model cannot hallucinate around;

2 The AgentGuardUtil

2.1 Architecture

Fig. 1 and Algo. 1 follow one turn through the verification harness. The turn is prepared in three steps. *First*, at the first turn of a conversation, `RUNTIMEPOLICYCOMPILER` translates the policy text Π into typed, executable rules $\mathcal{R}$ (§2.2); the result is cached under `hash( $\Pi$ )`, so later turns skip this step. *Second*, `EXTRACTTASKS` scans the new user message and appends each explicit request to a ledger L of outstanding asks, against which the completion gate (§2.4) later checks any claim of success. *Third*, the student proposes a draft a , i.e. a candidate answer for the turn, a spoken reply, tool calls, or both, which is neither shown to the driver nor executed until it passes verification. If the draft would mutate environment state and voting is enabled, the same prompt is sampled n times from the same LLM and `MODALVOTE` keeps the *action set* proposed most often (the multiset of tool calls with normalized arguments), a majority vote in the sense of self-consistency [1].

The draft then enters a verify-and-revise loop of at most R rounds, and each round separates two kinds of evidence. *First*, the deterministic gates of §2.4, among them the obligation engine of §2.3, produce *hard* findings F_{hard} , which stand as issued. *Second*, the same gates produce *advisory* candidates F_{adv} , which `CRITIC` (an LLM) may confirm or discard, but can neither extend with a finding of its own nor use to override a hard one. If the merged tiered list F is empty, the loop exits; otherwise `REVISE` re-prompts the student with F

Algorithm 1: VERIFIEDTURN($h, \mathcal{S}$)

```

1 if first turn for this policy  $\Pi$  then
2    $\mathcal{R} \leftarrow \text{RUNTIMEPOLICYCOMPILER}(\Pi, \mathcal{S})$ ;           // cached by
   hash( $\Pi$ )
3  $L \leftarrow L \cup \text{EXTRACTTASKS}(\text{new user msg})$ ;
4  $a \leftarrow \text{MODALVOTE}(\{\text{LLM}(h, \mathcal{S})\}_{1..n})$ ;
5 for  $i \leftarrow 1$  to  $R$  do
6    $F \leftarrow F_{\text{hard}}(a, h, \mathcal{R}) \cup \text{CRITIC}(F_{\text{adv}}(a, h, \mathcal{R}))$ ;
7   if  $F = \emptyset$  then break;
8    $a \leftarrow \text{REVISE}(h, a, F)$ ;
9 if  $a$  was revised and  $F_{\text{hard}}(a) \neq \emptyset$  then
10   $a \leftarrow \text{REVISE}_{\text{esc}}(h, a, F_{\text{hard}}(a))$ ;           // safety net
11 return SANITIZE( $a$ )
```

injected as an internal note, and the revision re-enters verification.

Two steps close the turn. *First*, because a revision can itself introduce a new defect, a safety net follows the loop: if the draft was revised and still carries hard findings, `REVISEesc` runs once more on those deterministic findings alone, optionally under a stronger *escalation* model. *Second*, `SANITIZE` strips the reply of markup that cannot be spoken, and the turn is released: verified tool calls reach the environment and the sanitized reply reaches the simulated driver.

2.2 Runtime Policy Compilation

At the first turn of a conversation, the policy text Π is compiled by an LLM into a rule set $\mathcal{R}$. Each rule $r = \langle \text{id}, \kappa, \mathcal{T}_r, \rho \rangle$ carries a type κ (*precondition*, *auto-action*, *confirmation*, *prohibition*, *disclosure*, *constraint*, *selection*), trigger tools $\mathcal{T}_r$ grounded against the live tool inventory, and an imperative requirement ρ . Compound clauses are split into atomic rules (one condition, one remedy). A second, focused

pass translates state-conditional rules into an *executable* form

$$e_r = \langle g, \varphi, \triangleleft, v, \omega \rangle, \quad (1)$$

where g is a read tool, φ a field-name pattern with one wildcard, $\triangleleft \in \{<, \leq, >, \geq, =, \neq, \exists, \nexists\}$, v a threshold, and $\omega(x) = \langle u, \beta(x) \rangle$ an obligation template binding the matched item x into the arguments of tool u . The compiler sees real signatures $u(p_1, \dots, p_m)$; an obligation whose arguments cannot be validated is dropped, degrading that rule to advisory, the engine can only *add* precision, never force an invalid call.

Example. To illustrate policy compilation, we consider the example of the policy “**When setting the air conditioning to ON, automatically close all windows if they are open more than 20%.**” first compiles to the typed rule $r_{011} = \langle 011, \text{auto-action}, \{\text{set_air_conditioning}\}, \rho \rangle$. The trigger set contains the tool whose invocation activates the rule, which is the remedy. The second pass grounds r_{011} into the executable form of Eq. (1):

$$e_{011} = \langle \text{get_vehicle_window_positions}, \\ \text{window_*_position}, >, 20, \\ \text{open_close_window}(\text{item}), 0 \rangle.$$

The executable form means: *fetch the window positions with g ; every returned field matching φ whose value exceeds $v=20$ binds its wildcard match (driver, passenger, ...) into the obligation template ω , validated against the declared enum of the window parameter (DRIVER, PASSENGER, ..., ALL). At runtime (§2.3), with observed positions 25/100/0/10, exactly two obligations survive, $\text{open_close_window}(\text{DRIVER}, 0)$ and $\text{open_close_window}(\text{PASSENGER}, 0)$, and are quoted to the student verbatim with their evidence, $\text{window_driver_position}=25 > 20$.*

2.3 Executable Obligations

The agent perceives the environment only through tool calls: *observation* tools report part of the environment state, and *action* tools change it. For every draft run, the engine answers one question: *given everything the agent has observed and the actions it now proposes, which further actions does the policy still require?* It proceeds in three steps:

- *Step 1: estimate the post-action environment state.* Let σ_g be the most recent observation returned by tool g , let $w_1, \dots, w_m$ be the actions executed after that observation, and let W_a be the actions the draft proposes. Each action may have changed, or would change, what was observed, so the engine applies all of them, in order, to the observation and obtains a *projected* state $\hat{\sigma} = \pi(\sigma_g; w_{1..m} \cdot W_a)$: its best estimate of the environment *after* the draft would execute. An action whose effect is predictable from a compiled rule is *simulated*, i.e. the affected state variable is set to the action’s argument value (an ALL-style argument affects every matching variable); an action whose effect is not predictable,

or that failed, marks the affected variables as unknown. Because rules are evaluated on this post-action estimate, a draft whose own actions would create a violation immediately produces the actions that repair it.

- *Step 2: compute the missing actions.*

A rule is *active* if one of its trigger tools was already used or appears in the draft ($\mathcal{R}_\triangleright = \{r : \mathcal{T}_r \cap (W_h \cup W_a) \neq \emptyset\}$).

$$\Omega(a, h) = \bigcup_{r \in \mathcal{R}_\triangleright} \{\omega(x) : \varphi \sim x, \hat{\sigma}(x) \triangleleft v\} \setminus (E_h \cup A_a). \quad (2)$$

For every active rule, and for every entity x (a window, a seat, a destination) whose projected value violates the rule’s condition, the engine fills the rule’s action template ω with x ; it then removes actions the agent has already performed (E_h) or already included in the draft (A_a). What remains, Ω , is exactly the set of actions the agent still owes.

- *Step 3: match the expected granularity.* If every entity violates the condition and the tool accepts an ALL-style argument, the per-entity actions collapse into one collective action, the granularity against which the benchmark scores intermediate states. The mirror check flags *excess*: actions on entities the condition exempts, and collective actions where only some entities qualify. Each finding quotes its actions ready to execute, together with the observed evidence, so the student no longer has to *recall* the policy — it only has to *transcribe* the correction.

Example. Continuing e_{011} : the windows were observed at 25/100/0/10 (driver, passenger, and the two rear windows), and the draft proposes only $\text{set_air_conditioning}(\text{on})$. That proposed action activates rule 011; no other action has run since the observation, so the projected state equals the observed one. The driver (25) and passenger (100) windows violate the > 20 condition, and no closing action exists in the history or the draft, so $\Omega = \{\text{open_close_window}(\text{DRIVER}, 0), \text{open_close_window}(\text{PASSENGER}, 0)\}$ — issued as a hard finding with the evidence $\text{window_driver_position}=25 > 20$. The projection matters in both directions. Had the draft instead proposed opening all windows to 50% with the air conditioning on, simulating it would put every window at 50 in the projected state and the finding would demand the closes — a violation the draft itself would create. Conversely, once the student adds the two closing actions, simulation drives both windows to 0, $\Omega = \emptyset$, and the draft is approved.

2.4 Tiered Verification and Variance Control

Around the obligation engine run 25 deterministic gates. Each gate checks one protocol the draft must respect, and each is decidable from the episode’s own artifacts: the transcript, the tool schemas, and prior observations. Each gate is a function that takes the draft a , the conversation history h , and the compiled rules $\mathcal{R}$, and returns findings, hard or advisory; the most important ones are summarized in Table 1.

Gate	What it checks	Example finding
<i>Identifier provenance</i>	Collects every opaque identifier in the draft’s tool calls and looks each one up among the identifiers returned by earlier observations. An identifier that was never observed blocks the offending call: the agent may <i>copy</i> identifiers, never <i>invent</i> them.	Routing to <code>loc_lux.222378</code> is allowed only if some earlier tool result returned that id — a deterministic anti-hallucination check.
<i>Schema validity</i>	Compares every argument of every proposed call against the tool’s declared parameter types and allowed values, before anything executes. An argument outside the declaration is rejected.	<code>open_close_window(window=FRONT)</code> is rejected because <code>FRONT</code> is not among the declared window values.
<i>Gather-before-act</i>	For every rule the draft activates, checks whether the observation its condition depends on has ever been made. If not, demands that observation — always a read-only call, never an action.	Rule 011 conditions on window positions, so <code>set_air_conditioning(on)</code> is held until <code>get_vehicle_window_positions</code> has been called (§2.2).
<i>Confirmation protocol</i>	For every proposed call the policy marks as requiring consent, searches the history for an intent question that the user affirmed. Without one, the call is blocked.	Opening a window beyond 25% while the air conditioning is on requires a question such as “shall I proceed?” and a “yes” before the action may execute.
<i>Completion</i>	Extracts the actions the draft’s reply claims to have performed and verifies that each was actually executed or is part of the draft itself. A claim without a matching action is rejected.	“I have closed all windows” with no <code>open_close_window</code> call is returned to the student.
<i>Refusal-read</i>	When the reply tells the user something cannot be done, checks whether the free observation that would answer the request was ever made; if not, demands that read first.	Replying “I cannot check the rear window” without ever calling <code>get_vehicle_window_positions</code> forces that read.
<i>Future-condition</i>	Detects that the request refers to a future moment (e.g. an arrival time) and checks that the draft answers it from unfiltered data evaluated at that moment, rather than filtering by the current moment.	“Will the charging station still be open when I arrive at 20:00?” must be answered from the station’s opening hours, not from an <code>open_now</code> filter that reflects the present.

Table 1: The principal deterministic gates. Each takes the draft a , the history h , and the compiled rules $\mathcal{R}$, and returns findings; all seven above issue *hard* findings, which stand as issued and always trigger revision.

Algorithm 2: $\text{VERIFYDRAFT}(a, h, \mathcal{R})$ — one verification step (the line $F_{\text{hard}} \cup \text{CRITIC}(F_{\text{adv}})$ of Algorithm 1)

```

1  $F_{\text{hard}} \leftarrow \emptyset$ ;  $F_{\text{adv}} \leftarrow \emptyset$ ;
2 foreach gate  $\gamma \in \Gamma$  do
3   //  $\Gamma$ : 25 deterministic gates, incl. the
   // obligation engine  $\Omega$  of §2.3
4    $(H, A) \leftarrow \gamma(a, h, \mathcal{R})$ ; // on internal error:  $(\emptyset, \emptyset)$ 
5    $F_{\text{hard}} \leftarrow F_{\text{hard}} \cup H$ ;  $F_{\text{adv}} \leftarrow F_{\text{adv}} \cup A$ ;
6 if  $F_{\text{adv}} \neq \emptyset$  then
7    $F_{\text{adv}} \leftarrow \text{CRITIC}(F_{\text{adv}}, h, S)$ ; // LLM critic: confirm
   // or discard only; never adds, never overrides
    $F_{\text{hard}}$ 
8 foreach  $f \in F_{\text{hard}} \cup F_{\text{adv}}$  do
9   if  $f$  re-demands an action set the student already revised away then
10    drop  $f$  and relax the gather gate // oscillation valve;
11 return  $F_{\text{hard}} \cup F_{\text{adv}}$ ; // tiered findings list  $F(a)$ 

```

Algorithm 2 details how the gates are combined into the single verification step that Algorithm 1 invokes once per revision round. *First*, every gate $\gamma \in \Gamma$ is run over the same triple $(a, h, \mathcal{R})$ and its two output sets are accumulated separately; a gate that raises an internal error contributes nothing, so a defective check can never block a well-formed draft. *Second*, if any advisory candidates were raised, the LLM critic adjudicates them as a batch against the transcript and the tool schemas: it may confirm or discard, but it may neither introduce a finding of its own nor overturn a hard one. *Third*, an *oscillation valve* scans the surviving findings and drops any that re-demand an action set the student has already revised away, relaxing the gather gate for the remainder of the turn; this prevents two gates from trading a draft back and forth

until the round budget R is exhausted. What the step returns is the tiered findings list $F(a)$.

Tiering is what keeps the harness’s own variance bounded. A hard finding is deterministically certain and always triggers revision, so the decisive checks never depend on a sampled judgment; an advisory candidate is uncertain by construction and reaches the student only if the critic confirms it against the episode’s artifacts. Because the critic is confined to filtering a set it did not generate, a bad sample can only cost a missed advisory, never a spurious correction or a suppressed hard finding, the guarantees carried by the teacher degrade gracefully rather than inverting.

3 Conclusion

AgentGuardUtil pairs an LLM student that drafts each turn with a deterministic teacher that checks every draft before it takes effect: the policy is compiled once into executable rules, each draft is evaluated against what was actually observed, including the state the environment would reach after the draft’s own actions, and the findings go back to the student until none remain. This addresses the three challenge classes directly: hallucinated identifiers are blocked (every outgoing identifier must have been observed), ambiguous requests are held behind confirmation checks, and forgotten policy side-effects come back as ready-to-execute actions.

References

- [1] Toufique Ahmed and Premkumar Devanbu. Better patching using llm prompting, via self-consistency. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1742–1746. IEEE, 2023.
- [2] Redouane Bouchekir and Mohand Cherif Boukala. Learning-based symbolic assume-guarantee reasoning for markov decision process by using interval markov process. *Innovations in Systems and Software Engineering*, 14(3):229–244, 2018.
- [3] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-verification reduces hallucination in large language models. In *Findings of the association for computational linguistics: ACL 2024*, pages 3563–3578, 2024.
- [4] Johannes Kirmayr, Lukas Stappen, and Elisabeth Andre. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 40599–40618, San Diego, California, United States, July 2026. Association for Computational Linguistics.